FIR Filter Design and Implementation

This project focused on the design and hardware implementation of a low-pass FIR digital filter. A low-pass filter passes frequency components below a chosen cutoff region and attenuates components above it. In the context of this project, the filter was designed so that the passband extended to 0.2π rad/sample, the stopband began at 0.23π rad/sample, and the stopband attenuation was at least 80 dB.

An FIR, or finite impulse response, filter computes each output sample as a weighted sum of the current input and a finite number of past input samples. If the filter coefficients are h[0], h[1], ..., h[N-1], then the output can be written as a sum of delayed input samples multiplied by those coefficients. This makes the FIR structure a natural match for hardware, since it can be implemented using a delay line, multipliers, and an adder network. FIR filters are also attractive because they are inherently stable and can be designed to have well-controlled frequency responses. For an FIR filter, the output is given by: $y[n] = \Sigma\, h[k]x[n-k]$, where h[k] are the filter coefficients and x[n-k] are the current and delayed input samples.

The project used MATLAB to design the filter and choose fixed-point formats, then implemented several hardware versions in Verilog. Those versions included a baseline direct-form design, parallel designs with L=2 and L=3 decomposition, a pipelined design, and a combined pipelined plus L=3 design. Synopsys Design Compiler was then used to compare the resulting area, timing, and power tradeoffs.

Figure 1: Project Flow Diagram

1. MATLAB FIR Design and Verilog Structure

The MATLAB portion of the project was used to design the FIR filter, evaluate its response, choose fixed-point formats, and generate the files needed for the hardware implementation. The design started from a Parks-McClellan equiripple FIR approach using firpm. That choice made sense for this project because the main design targets were frequency-domain constraints: a passband, a stopband, and a narrow transition band. The passband edge was set to 0.20 of Nyquist and the stopband edge was set to 0.23 of Nyquist, matching the required transition region. The initial trial used order 99 first, which corresponds to 100 taps, because the assignment said to start with a 100-tap low-pass filter and increase the number of filter taps if necessary. That first design did not meet the stopband requirement, so the order was increased until the stopband attenuation crossed 80 dB. The first 100-tap trial was not close enough to the

final target. It gave about 50.78 dB stopband attenuation and about 0.141 passband ripple, so the order had to be increased.

The final accepted floating-point design used 232 taps. That number came from the MATLAB loop that kept increasing the order until the measured stopband attenuation reached the target. In other words, 232 taps was the first design that satisfied the required attenuation with the chosen weighting. The final floating-point response achieved about 80.05 dB stopband attenuation and about 0.004785 passband ripple. This gave a filter that met the project spec while keeping the tap count as low as possible within that design flow.

Passband edge: wp = 0.20
Stopband edge: ws = 0.23
Stopband attenuation: Ast >= 80 dB

Filter order N = 231
Number of taps = N + 1 = 232

The unquantized magnitude response plot shows that the floating-point design met the required low-pass behavior before fixed-point effects were introduced. With the passband and stopband edge shown as red dotted vertical lines.

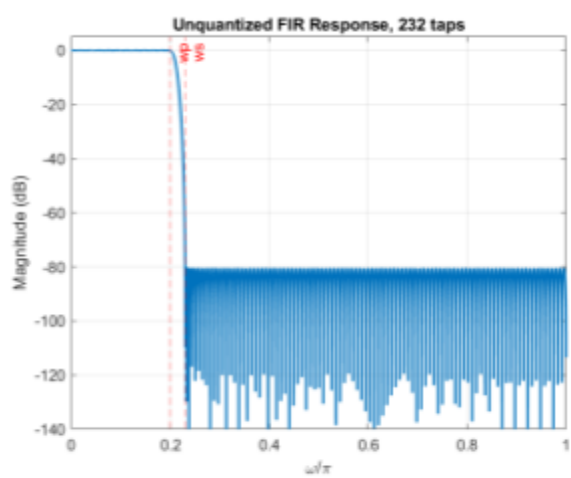

Figure 2: Unquantized freq. response with vertical markers at wp and ws

Initial design: order = 99, taps = 100
Passband ripple = 0.141453

Stopband attenuation = 50.78 dB

Final accepted design:
Order = 231
Taps = 232
Passband ripple = 0.004785
Stopband attenuation = 80.05 dB

After the floating-point design was accepted, the coefficients were quantized and a coefficient word-length sweep was performed. This was done to find the smallest practical coefficient width that still preserved the filter spec. The final coefficient format selected was Q1.21, which corresponds to a 22-bit signed coefficient width. Input and output samples were both kept in Q1.15 format. Those choices kept the input and output interface simple while still giving the coefficients enough precision to hold the 80 dB stopband target.

The reason Q1.21 was selected is that lower coefficient widths degraded the stopband too much, while larger widths did not provide enough additional benefit to justify the extra cost. So the final choice came from the sweep results, not from a fixed assumption at the start. The coefficient sweep covered widths from 16 to 24 bits. At the lower widths, the stopband attenuation dropped below the target even though the filter shape still looked reasonable overall. Moving to 22-bit coefficients kept the quantized response close enough to the floating-point design to satisfy the stopband requirement, so that format was used in the final RTL. For example, the lower-width coefficient trials reduced hardware cost but degraded the stopband too much, while the final 22-bit coefficient format preserved the required attenuation and kept the quantized response close to the floating-point design.

$$H_{q,int}[n] = \mathrm{round}(h[n] * 2^{(Bcoef-1)})$$
$$h_q[n] = h_{q,int}[n] / 2^{(Bcoef-1)}$$

- $H_{q,int}[n]$ are the quantized integer coefficients

Chosen coefficient format: Q1.21
Chosen coefficient width: 22 bits signed

MATLAB was also used to calculate a safe accumulator width and to generate a fixed-point golden reference output using an explicit integer multiply-accumulate model. This part was important because the RTL was not just a floating-point translation of the filter. The hardware had to use finite word lengths, rounding, shifting, and saturation. The MATLAB reference model matched that behavior directly, which made it possible to compare the intended arithmetic behavior against the hardware implementation.

The Verilog structure followed the same fixed-point assumptions used in MATLAB. All architectures used the same top-level idea: an input delay line stores the most recent samples,

those delayed samples are multiplied by constant coefficients, the results are summed, then the accumulator is rounded, shifted, and saturated into the final output width. The main difference between the Verilog versions is how the accumulation path is organized. The baseline version uses a direct-form structure. The parallel versions reorganize the coefficients into phase branches. The pipelined versions add registers to shorten the critical path. This kept the arithmetic model consistent across all versions while making the architecture tradeoffs easier to compare.

Another useful part of the flow was generating include files directly from the MATLAB export files. That kept the Verilog coefficient files, tap count, coefficient width, accumulator width, and shift amount consistent with the MATLAB design choices, which reduced the chance of mismatches between the software and hardware stages.

2. Frequency Response, Quantization Effect, and Overflow Handling

The original floating-point filter achieved the target stopband attenuation while maintaining the intended low-pass shape. After quantization, the frequency response stayed very close to the floating-point response. The final quantized design still met the stopband target, so coefficient quantization did not cause a failure of the specification. The quantized response did show some small deviation from the floating-point curve, but the overall passband and stopband behavior remained close enough for the design goal. The floating-point and quantized responses were also very close numerically. In the final design summary, the floating-point filter had about 80.0485 dB stopband attenuation and about 0.00478490 passband ripple, while the quantized filter had about 80.0462 dB stopband attenuation and about 0.00478305 passband ripple. This means the final coefficient quantization changed the response only slightly and did not push the filter out of spec. In practice, the quantized filter still behaved like the same low-pass design, with only the expected small finite-word-length error.

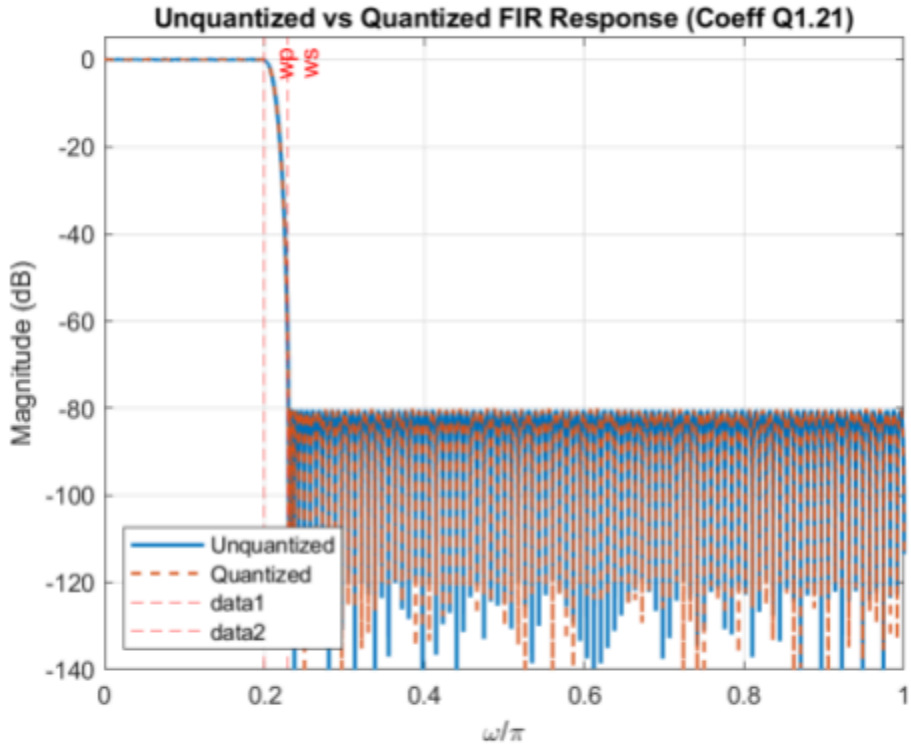

Figure 4: Unquantized vs Quantized Magnitude response

A coefficient-width sweep was useful because it showed the tradeoff between coefficient precision and hardware cost. Using too few bits caused worse stopband attenuation and larger response error, while using more bits increased hardware cost. The chosen coefficient width gave a good balance between maintaining the filter specification and avoiding unnecessary cost. The sweep showed that coefficient precision becomes important once the stopband target is pushed to 80 dB. A filter that looks acceptable at lower precision may still miss the actual stopband target. For example, the lower-width trials produced weaker stopband rejection, while the final selected width preserved the target more closely. In the sweep, 16-bit coefficients gave about 67.98 dB stopband attenuation, 17-bit gave about 69.51 dB, 18-bit gave about 74.08 dB, and 19-bit gave about 77.09 dB.

Overflow handling was considered explicitly in both MATLAB and Verilog. MATLAB was used to estimate a safe accumulator width from the maximum possible input magnitude and the sum of the absolute coefficient values. This gave a worst-case bound for the signed accumulator. That width was then used in the fixed-point model and later carried into the RTL. The final accumulator width was 39 bits. That number came from the worst-case fixed-point accumulation bound, not from guessing. The fixed-point reference run also showed that the chosen word lengths were sufficient for the test stimulus used in the project. The worst-case accumulator bound computed from the coefficient sum was 149255061214, while the largest accumulator magnitude actually seen during the fixed-point run was 34201578422. In addition, the summary reported 0 accumulator saturations and 0 output saturations. This gave confidence that the

selected 39-bit accumulator width and 21-bit output shift were consistent with the intended arithmetic path for the chosen test input.

$$max_abs_acc = (2^{(Bin - 1)} - 1) * \Sigma|h_q, int[n]|$$

This leads to a required accumulator width of ACC_W = 39 bits to hold max_abs_acc

The output shift amount also came directly from the selected fixed-point formats. Since the input was Q1.15 and the coefficient was Q1.21, the raw product had 15 + 21 = 36 fractional bits. The output was required back in Q1.15, so the accumulator result had to be shifted right by 21 bits before saturation to the 16-bit output format. That is where the final shift amount came from.

$$SHIFT_BITS = (Bin - 1) + (Bcoef - 1) - (Bout - 1)$$
$$SHIFT_BITS = 15 + 21 - 15 = 21$$

This also matches the design summary, which reported 36 product fractional bits and a final shift of 21 bits to return the accumulator result to Q1.15 output format.

A fixed-point integer reference model was then built to simulate the full hardware arithmetic path, including signed multiplication, accumulation, rounding, shifting, and output saturation. This showed that the selected accumulator width was sufficient for the chosen stimulus and that the final output behavior matched the intended fixed-point model. In the hardware RTL, rounding is applied before the output shift and saturation limits are enforced before the final output register. This kept the arithmetic consistent across the software and hardware sides of the project.

Another useful part of the MATLAB flow was the export of fixed-point verification vectors. The script wrote the quantized input stimulus to x_input_q15.txt and the expected fixed-point output to y_golden_q15.txt. These files were produced by the same integer reference model that applied the chosen coefficient format, accumulator width, rounding rule, output shift, and saturation behavior. That means the vectors reflect the exact arithmetic the RTL was intended to implement, not just the floating-point filter response. Including these files in the workflow made it possible to compare the hardware output against a known fixed-point reference and helped connect the MATLAB design stage to the Verilog implementation stage. The test stimulus used for the fixed-point reference model was also chosen to exercise different parts of the filter response. The MATLAB script used three tones: 0.10, which lies in the passband; 0.21, which lies near the transition region; and 0.35, which lies in the stopband. That made the fixed-point check more meaningful, since the golden vectors were not based on only one passband tone.

3. FIR Architectures

Five FIR architectures were implemented and synthesized. All of them use the same fixed-point assumptions and the same coefficient set, so the differences in the hardware results mainly come from structure rather than from changing the filter specification.

Baseline:
The baseline version is the direct-form reference design. A full input delay line stores the most recent 232 samples. Each sample is multiplied by its corresponding coefficient, and all terms are summed into one accumulator. This is the simplest version structurally and serves as the reference case for timing, area, and power comparison. The baseline is useful because it shows what happens if the filter is implemented in the most straightforward way, without any effort to reduce the critical path.

Parallel L=2:
The L=2 design uses reduced-complexity parallel processing by separating the coefficients into two phases: even-index coefficients and odd-index coefficients. Two branch sums are formed using those phase coefficient sets, and then the branches are combined into the final accumulated result. To keep the local combinational depth shorter, each branch was also broken into two smaller internal groups before being combined. The purpose of this structure was to shorten the longest summation path compared to a plain direct-form chain, while keeping the architecture simpler than the L=3 case.

Parallel L=3:
The L=3 design extends the same idea by separating the coefficients into three branches based on index modulo 3. Three phase sums are formed and then combined. To improve speed over a simple branch sum, each branch was divided into four smaller local groups, and those local groups were reduced in a more balanced way before the final branch combine. This gave the tool a shorter and more balanced combinational structure than the original one-branch accumulation style.

Pipelined:
The pipelined architecture keeps the same overall filter behavior as the baseline but inserts registers inside the long accumulation path. Instead of one long sum, the taps are divided into several smaller groups, those partial sums are registered, then combined again through additional pipeline stages. This reduces the combinational depth seen in any one clock period and was intended to improve clock speed. This version does not change the filter itself. It changes only how the sum is scheduled across cycles.

Pipelined + Parallel L=3:
The combined architecture starts from the L=3 branch decomposition, then adds multiple pipeline cuts inside the branch reduction and branch-combine path. This version uses local subgroup sums, registered partial branch reductions, registered branch sums, and a registered final combine. The purpose of this version was to push speed as far as possible while still using the L=3 parallel structure. Out of the five architectures, this one was the most timing-focused.

An important part of the hardware work was that the first synthesis results did not show the timing order that was expected. Early versions of the parallel and pipelined architectures were not giving enough timing improvement, and the fastest architectures did not clearly separate themselves from the simpler ones. To fix that, the reduction paths in the L=2 and L=3 versions were reorganized into smaller local sums, and additional pipeline cuts were added to the combined pipelined plus L=3 version. That refinement step was important in getting the final timing order closer to the expected trend.

4. Detailed Hardware Implementation Results

All architectures were synthesized in Synopsys Design Compiler using the same 10 ns target clock constraint as a common comparison point. Even though none of the designs fully met 10 ns timing closure, the slack values still allowed a consistent speed comparison and allowed an estimated achievable clock period and clock frequency to be calculated.

Instead of choosing a coefficient width by guesswork, the MATLAB script tested several candidate widths and re-measured the quantized response each time. Total cell area came from the area report, worst slack came from the timing report under the 10 ns target clock, and dynamic power came from the power report. The estimated minimum clock period was not reported directly by the tool in the exact form used in the summary table. That number was calculated from the 10 ns target plus the magnitude of the negative slack. Then the estimated maximum clock frequency was computed from that estimated minimum clock period.

$$T_{min} = T_{constraint} + |slack|$$
$$f_{max} = 1 / T_{min}$$

For pipelined + parallel L=3:

$$T_{min} \approx 10\text{ ns} + 10.46\text{ ns} = 20.46\text{ ns}$$
$$f_{max} \approx 1 / 20.46\text{ ns} = 48.88\text{ MHz}$$

For example, the final pipelined plus L=3 architecture had a worst slack of -10.46 ns under a 10 ns target. That means the estimated minimum clock period is about 20.46 ns, and the estimated maximum clock frequency is about 48.88 MHz. The same method was used for the other architectures so the comparison stayed consistent.

The final hardware results are listed below.

Architecture: Baseline
Total cell area: 357213
Worst slack: -46.02 ns
Estimated minimum clock period: 56.02 ns

```
Estimated maximum clock frequency: 17.85 MHz
Total dynamic power: 846.4089 uW

Architecture: Parallel L=2
Total cell area: 185310
Worst slack: -34.68 ns
Estimated minimum clock period: 44.68 ns
Estimated maximum clock frequency: 22.38 MHz
Total dynamic power: 2.9167 mW

Architecture: Parallel L=3
Total cell area: 185172
Worst slack: -34.60 ns
Estimated minimum clock period: 44.60 ns
Estimated maximum clock frequency: 22.42 MHz
Total dynamic power: 2.9160 mW

Architecture: Pipelined
Total cell area: 277336
Worst slack: -11.20 ns
Estimated minimum clock period: 21.20 ns
Estimated maximum clock frequency: 47.17 MHz
Total dynamic power: 288.7574 uW

Architecture: Pipelined + Parallel L=3
Total cell area: 249064
Worst slack: -10.46 ns
Estimated minimum clock period: 20.46 ns
Estimated maximum clock frequency: 48.88 MHz
Total dynamic power: 3.7255 mW
```

From a timing perspective, the final speed order was:

1. Pipelined + Parallel L=3
2. Pipelined
3. Parallel L=3
4. Parallel L=2
5. Baseline

This is close to the expected trend, where the most aggressively pipelined architecture performs best in speed and the baseline performs worst. The L=3 version slightly outperformed the L=2 version, which is also consistent with the expectation that more aggressive structural decomposition can help timing when implemented carefully.

From an area perspective, the parallel L=2 and L=3 designs were the smallest. That result makes sense because those two versions reorganized the arithmetic structure without inserting as many registers as the pipelined versions. From a power perspective, the pipelined-only design had the lowest dynamic power, while the fastest combined pipelined plus L=3 design had the highest dynamic power. This shows that improving speed did not automatically improve every other metric and that the fastest design came with a clear power cost.

One of the last timing improvements came from adding another pipeline cut inside the pipelined plus L=3 reduction path. Before that change, its slack was about -12.93 ns under the 10 ns target, which still left it slightly behind the pipelined-only version. After the extra pipeline stage was added, the pipelined plus L=3 slack improved to about -10.46 ns, which moved it ahead of the pipelined design at about -11.20 ns.

5. Analysis and Conclusion

The final results show a clear architecture tradeoff. The baseline design was the slowest and also had the largest area in the final comparison. The parallel L=2 and L=3 versions reduced area significantly and improved timing compared to the baseline, but they still did not approach the performance of the pipelined versions. The pipelined-only architecture gave a very strong timing improvement and also had the lowest dynamic power, making it the best timing-power tradeoff among the tested designs. The combined pipelined plus L=3 architecture achieved the best overall speed and ended up as the fastest architecture in the project, but this came at the cost of much higher dynamic power.

A 10 ns synthesis constraint was used for all designs as a common target, but none of the designs fully closed timing at that target. Even though none of the designs met 10 ns, the results still show a clear timing ranking and clear architecture tradeoffs. The point of the comparison was to see which architecture trends moved the design in the right direction. In that sense, the comparison was still successful. The final fastest design was the pipelined plus L=3 architecture, which matches the general expectation that more aggressive structural parallelism plus pipelining should help timing. At the same time, the pipelined-only version ended up as the strongest balanced design when both timing and power are considered together.

The main tradeoffs are clear when each architecture is viewed in terms of speed, area, and power. The baseline design is the simplest and most direct implementation, but it is also the slowest. The L=2 and L=3 parallel versions both cut area and improve timing compared to the baseline, with L=3 giving a small additional speed advantage over L=2. The pipelined design delivers a much larger timing improvement and also has the lowest dynamic power, so it gives the best overall balance between speed and power. The pipelined plus L=3 design reaches the highest speed of all five architectures, but that extra performance comes at the cost of the highest dynamic power. This makes the overall tradeoff clear: the fastest structure is not the most power-efficient one, while the most balanced structure is not the absolute fastest.

Overall, the project shows how FIR implementation results depend heavily on architecture choice, not just coefficient values. The MATLAB flow was useful for establishing a correct filter response and safe fixed-point arithmetic, while the Verilog and Design Compiler flow showed how timing, area, and power shift as the structure moves from a direct-form baseline to more parallel and more deeply pipelined versions.